
1. Singleton Pattern

Ensures a class has only one instance and provides a global point of access to it.

When to Use:

- When you need exactly one instance of a class (e.g., database connection, logger, configuration manager).
- When you want to control access to shared resources.

Example:

```
public class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("Hello from Singleton!");
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Singleton singleton = Singleton.getInstance();
        singleton.showMessage(); // Output: Hello from Singleton!
    }
}
```

2. Command Pattern

Encapsulates a request as an object, allowing parameterization of clients with queues, requests, and operations.

When to Use:

- When you need to decouple the sender of a request from the object that performs the action.
- When you want to support undo/redo functionality or queue requests.

Example:

```
// Command interface
interface Command {
    void execute();
}

// Receiver
class Light {
    void on() {
        System.out.println("Light is ON");
    }

    void off() {
        System.out.println("Light is OFF");
    }
}

// Concrete Command
class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.on();
    }
}
```

```
// Invoker
class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Light light = new Light();
        Command lightOn = new LightOnCommand(light);

        RemoteControl remote = new RemoteControl();
        remote.setCommand(lightOn);
        remote.pressButton(); // Output: Light is ON
    }
}
```

3. Decorator Pattern

Adds behavior to objects dynamically without affecting the behavior of other objects from the same class.

When to Use:

- When you want to add responsibilities to objects at runtime.
- When subclassing is impractical (e.g., too many combinations of features).

Example:

```
// Component interface
interface Coffee {
    int cost();
}

// Concrete Component
class SimpleCoffee implements Coffee {
    @Override
    public int cost() {
        return 5;
    }
}

// Decorator
class MilkDecorator implements Coffee {
    private Coffee coffee;

    public MilkDecorator(Coffee coffee) {
        this.coffee = coffee;
    }

    @Override
    public int cost() {
        return coffee.cost() + 2;
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Coffee coffee = new SimpleCoffee();
        System.out.println(coffee.cost()); // Output: 5

        Coffee milkCoffee = new MilkDecorator(coffee);
        System.out.println(milkCoffee.cost()); // Output: 7
    }
}
```

4. Builder Pattern

Separates the construction of a complex object from its representation.

When to Use:

- When the object creation process is complex or involves multiple steps.
- When you want to create different representations of the same object.

Example:

```
class Pizza {
    private String dough;
    private String sauce;
    private String topping;

    public void setDough(String dough) {
        this.dough = dough;
    }

    public void setSauce(String sauce) {
        this.sauce = sauce;
    }

    public void setTopping(String topping) {
        this.topping = topping;
    }

    @Override
    public String toString() {
        return "Pizza with " + dough + ", " + sauce + ", and " + topping;
    }
}
```

```
class PizzaBuilder {
    private Pizza pizza;

    public PizzaBuilder() {
        pizza = new Pizza();
    }

    public PizzaBuilder setDough(String dough) {
        pizza.setDough(dough);
        return this;
    }

    public PizzaBuilder setSauce(String sauce) {
        pizza.setSauce(sauce);
        return this;
    }

    public PizzaBuilder setTopping(String topping) {
        pizza.setTopping(topping);
        return this;
    }

    public Pizza build() {
        return pizza;
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Pizza pizza = new PizzaBuilder()
            .setDough("thin crust")
            .setSauce("tomato")
            .setTopping("cheese")
            .build();

        System.out.println(pizza); // Output: Pizza with thin crust, tomato, and cheese
    }
}
```

5. Factory Pattern

Creates objects without specifying the exact class of the object that will be created.

When to Use:

- When you want to delegate object creation to a separate method or class.
- When the exact type of the object is determined at runtime.

Example:

```
interface Animal {
    void speak();
}

class Dog implements Animal {
    @Override
    public void speak() {
        System.out.println("Woof!");
    }
}

class AnimalFactory {
    public Animal getAnimal(String type) {
        if (type.equals("dog")) {
            return new Dog();
        }
        return null;
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        AnimalFactory factory = new AnimalFactory();
        Animal animal = factory.getAnimal("dog");
        animal.speak(); // Output: Woof!
    }
}
```

6. Abstract Factory Pattern

Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

When to Use:

- When you need to create families of related objects.
- When you want to enforce consistency among products.

Example:

```
interface Button {
    void paint();
}

interface Checkbox {
    void paint();
}

class WinButton implements Button {
    @Override
    public void paint() {
        System.out.println("Windows Button");
    }
}

class WinCheckbox implements Checkbox {
    @Override
    public void paint() {
        System.out.println("Windows Checkbox");
    }
}

interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}
```



```
class WinFactory implements GUIFactory {
    @Override
    public Button createButton() {
        return new WinButton();
    }

    @Override
    public Checkbox createCheckbox() {
        return new WinCheckbox();
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        GUIFactory factory = new WinFactory();
        Button button = factory.createButton();
        button.paint(); // Output: Windows Button
    }
}
```

7. Facade Pattern

Provides a simplified interface to a complex subsystem.

When to Use:

- When you want to provide a simple interface to a complex system.
- When you want to decouple clients from subsystem components.

Example:

```
class CPU {
    void processData() {
        System.out.println("Processing data...");
    }
}

class Memory {
    void load() {
        System.out.println("Loading memory...");
    }
}

class ComputerFacade {
    private CPU cpu;
    private Memory memory;

    public ComputerFacade() {
        cpu = new CPU();
        memory = new Memory();
    }

    public void start() {
        cpu.processData();
        memory.load();
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        ComputerFacade computer = new ComputerFacade();
        computer.start();
        // Output:
        // Processing data...
        // Loading memory...
    }
}
```

8. Proxy Pattern

Provides a surrogate or placeholder for another object to control access to it.

When to Use:

- When you want to add a layer of control over access to an object (e.g., lazy initialization, access control, logging).
- When you want to defer the creation of expensive objects until needed.

Example:

```
interface Image {
    void display();
}

class RealImage implements Image {
    private String filename;

    public RealImage(String filename) {
        this.filename = filename;
        loadFromDisk();
    }

    private void loadFromDisk() {
        System.out.println("Loading " + filename);
    }

    @Override
    public void display() {
        System.out.println("Displaying " + filename);
    }
}
```

```
class ProxyImage implements Image {
    private RealImage realImage;
    private String filename;

    public ProxyImage(String filename) {
        this.filename = filename;
    }

    @Override
    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename);
        }
        realImage.display();
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Image image = new ProxyImage("test.jpg");
        image.display(); // Output: Loading test.jpg, Displaying test.jpg
    }
}
```

These patterns help solve common design problems and improve code maintainability, scalability, and flexibility.